Project 1 Report - Team 27

Course: CS6.401 Software Engineering (Spring 2026)

Project: Reverse Engineering and Refactoring Apache Roller

Repository: Apache Roller (https://github.com/serc-courses/project-1-team-27)

---

1. Abstract

This report documents the reverse engineering, design smell analysis, refactoring, and comparative evaluation performed on Apache Roller. We focused on three subsystems (Weblog and Content, User and Role Management, Search and Indexing), built UML models, identified design smells with tool evidence, executed class-level refactorings that preserve behavior, and re-ran metrics to verify impact. We also document LLM usage, including manual vs LLM comparisons, an LLM-based refactoring pipeline, agentic refactoring suggestions, and a comparative study of manual, LLM-assisted, and agentic refactorings.

The report is structured so that a reader can trace each task from inputs (tool outputs, UML, source code) to outputs (design smells, refactorings, metrics changes), including assumptions, methodology, and verification steps.

---

2. Project Context and Scope

2.1 Target System

Apache Roller is a mature Java-based multi-user blog platform supporting content authoring, role-based access, and Lucene-based search. The repository was analyzed in its provided structure and build system (Maven).

2.2 Focused Subsystems

1. Weblog and Content Subsystem
2. User and Role Management Subsystem
3. Search and Indexing Subsystem

2.3 Scope Boundaries

- Only key classes for the above subsystems were modeled in UML.
- We focused on design-level smells (system- and class-level), not only method-level code smells.
- Refactorings were class-level changes that preserve behavior.

2.4 Methodology Overview

Across tasks we followed a consistent workflow:

1. Identify scope and key classes from source packages and documentation.
2. Build UML models to capture structure, dependencies, and responsibility allocation.
3. Run static analysis tools and extract evidence for design smells and metrics.
4. Propose refactorings aligned with SOLID principles and design patterns.
5. Implement manual refactorings only when allowed, keeping behavior unchanged.
6. Re-run metrics and compare before/after impact.
7. Document LLM usage, limitations, and manual validation steps.

---

3. Task 1 - Architectural Mapping and Design Recovery

3.1 Identified Classes and Documentation

Detailed documentation for each subsystem and key class is in:

- Search and Indexing: docs/project1/Task 1/SEARCH-INDEXING-SUBSYSTEM-DOCUMENTATION.md
- User and Role Management: docs/project1/Task 1/USER-ROLE-MANAGEMENT-DOCUMENTATION.md
- Weblog and Content: docs/project1/Task 1/WEBLOG-CONTENT-SUBSYSTEM-DOCUMENTATION.md

3.2 UML Diagrams

PlantUML diagrams used for modeling:

- Search and Indexing UML: docs/project1/Task 1/uml/search-indexing-subsystem.puml
- User and Role Management UML: docs/project1/Task 1/uml/user-role-management.puml
- Weblog and Content UML: docs/project1/Task 1/uml/Weblog-and-Content-Subsystem.puml

3.3 Observations and Comments

Strengths, weaknesses, and architectural observations are documented in:

- docs/project1/Task 1/OBSERVATIONS-AND-COMMENTS.md

3.4 Methodology (Task 1)

Steps followed: 1. Mapped package structure and selected subsystem boundaries from the Roller codebase.

2. Located core interfaces, managers, POJOs, and UI components in each subsystem.
3. For each class, recorded responsibilities, key methods, and interactions.
4. Built PlantUML diagrams showing inheritance, association, and dependency edges.
5. Cross-checked UML against code to avoid missing key relationships.
6. Summarized strengths/weaknesses and noted assumptions.

3.5 Manual vs LLM Analysis (Task 1 Requirement)

Scope: Business layer of the Weblog subsystem.

- Manual analysis time: 8 hours.
- LLM analysis time: 30 minutes.
- LLM used: Claude 3.5 Sonnet.
- Findings and comparison: docs/project1/Task 1/Manual_Vs_LLM.md

Key takeaway: LLM output was faster and good at pattern recognition, but manual analysis provided concrete implementation details and validation.

---

4. Task 2 - Smells and Metrics Analysis

4.1 Task 2A - Design Smells

We identified 7 design smells with evidence from Designite and SonarQube and correlated them with UML models. Full details and evidence are in:

- docs/project1/Task 2A/Task2A-DesignSmells.md

Smells summarized:

1. Cyclic-Dependent Modularization (security and domain types)
2. Weblog as a Hub Object (hub-like modularization)
3. URLStrategy mixing concerns (search, rendering, navigation)
4. Duplicated search-result responsibilities
5. Broken theme/template hierarchy
6. Imperative abstraction in RollerUserDetailsService

7. Unnecessary abstraction in SingletonHolder

4.2 Methodology (Task 2A)

Steps followed:

1. Ran Designite Java and SonarQube on the Roller codebase.
2. Extracted candidate design smells from reports.
3. Validated each smell against UML relationships and source code structure.
4. Focused on system-level violations (cyclic dependencies, hub classes, broken hierarchies).
5. Documented evidence with references to the tool outputs and UML diagrams.

4.3 Task 2B - Code Metrics Analysis

Tools used: Checkstyle, Designite Java, CK metrics tool.

Selected metrics: CC, Method Length, Fan-Out Complexity, WMC, DIT, LCOM.

- docs/project1/Task 2B/Task2B-Code-Metrics.md

Implication summary:

- High CC and long methods in servlets and utilities are refactoring candidates.
- High fan-out and WMC in central managers and domain classes align with hub-like smells.
- Moderate DIT combined with high WMC indicates fragile inheritance in templates and URL strategies.
- LCOM suggests some domain types combine unrelated concerns.

4.4 Methodology (Task 2B)

Steps followed:

1. Collected metric reports using Checkstyle, Designite, and CK.
2. Selected six metrics to cover size, complexity, coupling, inheritance, and cohesion.
3. Identified outliers (top violations) and cross-checked between tools.
4. Interpreted each metric in terms of maintainability and refactoring priority.

---

5. Task 3 - Refactoring

5.1 Task 3A - Manual Refactoring

We refactored all 7 design smells with class-level changes and preserved behavior.

- Detailed changes and commit references: docs/project1/Task 3A/Task3A-Refactoring-Complete.md

Highlights:

- Extracted AuthorizationService to break permission cycles.
- Introduced WeblogService and WeblogConfig to reduce Weblog hub responsibilities.
- Segregated URLStrategy into focused interfaces (ISP).
- Removed SearchResultMap and unified search result handling.
- Simplified and flattened theme/template hierarchy.
- Split RollerUserDetailsService into focused loaders.
- Removed SingletonHolder abstraction.

Behavior verification: mvn install success (per Task 3A notes).

5.1.1 Methodology (Task 3A)

Steps followed:

1. For each design smell, identified the minimal class-level refactoring that reduces coupling or clarifies responsibilities.
2. Opened or referenced a GitHub issue before/after refactoring for traceability.
3. Applied refactorings with backward compatibility where public APIs were involved.
4. Kept changes behavior-preserving by delegating to existing managers/services.
5. Verified compile and tests using the existing build pipeline.

5.2 Task 3B - Post-Refactoring Metrics

We re-ran the same metrics and compared before vs after. - docs/project1/Task 3B/Task3B-Metrics-Analysis.md

Key improvements:

- Reduced coupling and WMC for Weblog and related classes.
- Reduced fan-out and simplified search result handling.

- DIT reduction in template hierarchy.
- Some trade-offs: total LOC increased due to new service/value object classes.

5.2.2 Baseline vs Refactored Comparison

SonarQube was used to support design smell identification and to validate refactoring impact. We used its rules on complexity, coupling, and duplication as evidence, and cross-checked findings with Designite and UML models.

How SonarQube was used:

1. Scanned the Roller codebase with SonarQube using the same configuration before and after refactoring.
2. Collected issues related to complexity, dependency cycles, and code duplication.
3. Mapped issues to the selected design smells and recorded evidence in Task 2A and Task 3B documentation.
4. Used issue lists to justify refactoring targets and compare before/after quality trends.

Screenshot placeholders:

- SonarQube project overview dashboard

Figure 1: alt text

  –
- SonarQube Technical Debt
  –

Baseline branch: main Refactored branch: task3_Akshat

Comparison sources:

- Metrics: Task 2B (baseline) vs Task 3B (refactored)
- SonarQube: baseline scan vs refactored scan
- Design smells: Task 2A vs Task 3A documentation

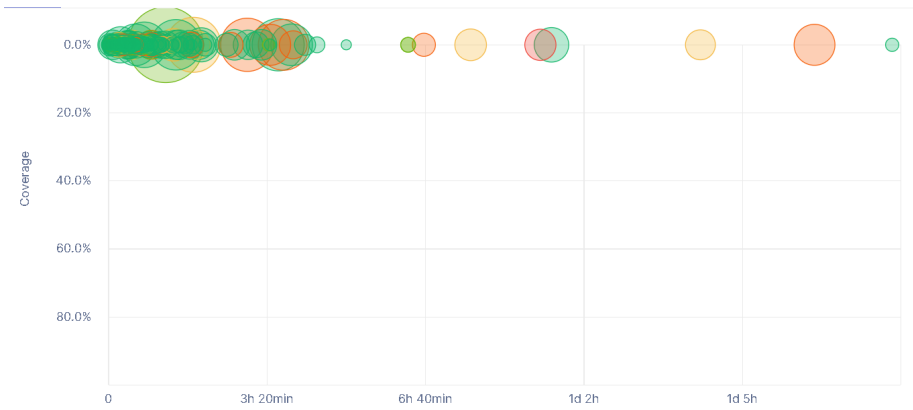


Figure 2: alt text

Screenshot placeholders:

- Refactored SonarQube dashboard

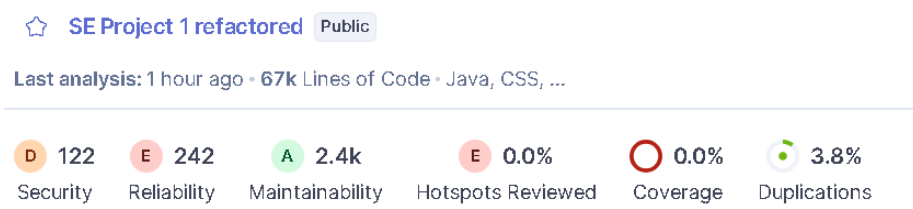


Figure 3: alt text

  –

- Baseline issues list
  –

5.2.1 Methodology (Task 3B)

Steps followed:

1. Re-ran the same tools and configurations as Task 2B.
2. Compared the same classes and methods from baseline to refactored state.
3. Documented improvements and trade-offs, including cases where a metric worsened while another improved.

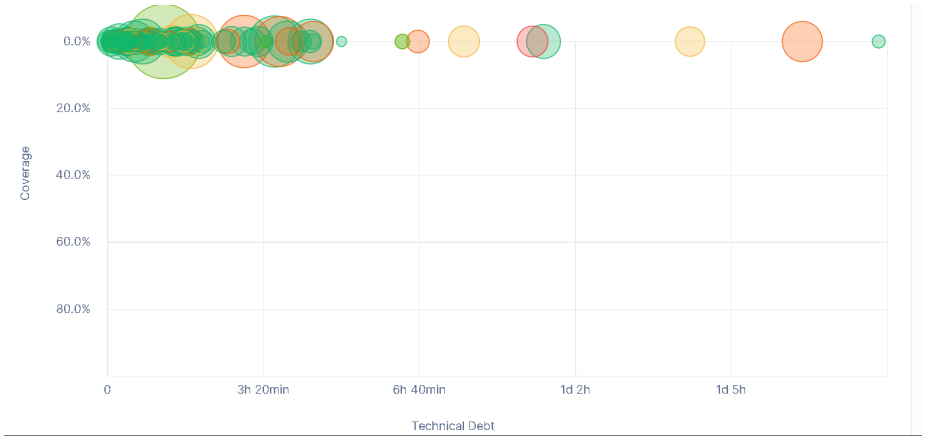


Figure 4: alt text

5.3 Task 3C - Automated LLM Refactoring Pipeline (Documented Only)

We designed and documented an LLM-based pipeline that detects smells, proposes refactorings, and generates PRs (not applied to the codebase).

- docs/project1/Task 3C/REFACTORING_ANALYSIS.md

Relevant branches:

- Task3C_Parv: Implements the automated pipeline for detection, refactoring suggestions, and PR creation using access tokens and automation.
- automated-refactoring/20260209-222604: LLM-generated reference output branch used for comparison and documentation.

Pipeline highlights:

- Smell detection on a subset of files with confidence scoring.
- Refactoring suggestions with before/after diffs.
- PR generation concept with detailed metadata and metrics.
- Build is not successful for this automated refactoring of the code.

5.3.1 Methodology (Task 3C)

Steps followed:

1. Defined pipeline stages: detect smells, refactor using LLM, generate PR with metadata.
2. Limited analysis to a subset of files to control cost and context size.

3. Produced refactoring diffs for documentation only (not merged into codebase).
4. Recorded pipeline outputs for reporting and reproducibility.

---

6. Task 4 - Agentic Refactoring on a Single File (Documented Only)

Agentic tool used: GitHub Copilot (Claude) in VS Code agentic workflow.

Files analyzed (4): LuceneIndexManager, WeblogEntry, SearchOperation, User.

- Summary: docs/project1/Task 4/Summary.md
- Workflow: docs/project1/Task 4/README.md
- Detailed analyses:
  - docs/project1/Task 4/File1_LuceneIndexManager_Analysis.md
  - docs/project1/Task 4/File2_WeblogEntry_Analysis.md
  - docs/project1/Task 4/File3_SearchOperation_Analysis.md
  - docs/project1/Task 4/File4_User_Analysis.md

6.1 Methodology (Task 4)

Steps followed:

1. Selected four representative files across subsystems.
2. Ran agentic analysis for smell identification, planning, and refactoring snippets.
3. Captured suggested refactorings and compared them with manual insights.
4. Documented outputs only; no code changes were merged from the agent.

---

7. Task 5 - Comparative Refactoring Analysis

We compared manual, LLM-assisted, and agentic refactoring approaches for three selected smells. - docs/project1/Task5/comperative_analysis.md

Summary findings:

- Manual refactoring provided best architectural alignment and test stability.
- LLM-assisted refactoring was fast but required manual corrections for design quality.

- Agentic refactoring provided rich suggestions but sometimes over-engineered the solution.

7.1 Methodology (Task 5)

Steps followed:

1. Selected three smells that cover different design issues.
2. Refactored each smell in three modes: manual, single-prompt LLM, agentic.
3. Evaluated each approach along clarity, conciseness, design quality, faithfulness, and architectural impact.
4. Recorded where automation helped and where human judgment was required.

---

8. LLM Usage Documentation

8.1 LLMs and Tools Used

- Claude 3.5 Sonnet (Task 1 LLM-assisted analysis)
- GitHub Copilot (Claude) (Task 4 agentic refactoring)
- GPT-4 Turbo (Task 5 LLM-assisted refactoring comparison)
- Cursor with Claude 3.5 Sonnet (Task 5 agentic comparison)

8.2 LLM Usage by Task (summary)

Task	Tool(s)	Output	Notes
1	Claude / ChatGPT	PlantUML patterns	Applied to subsystem diagrams
2	Designite, Checkstyle, SonarQube + ChatGPT	Run commands, smell mappings	Team ran tools and validated results
3	Gemini, GitHub + ChatGPT	Key handling, PR automation examples	Used for documented experiments (no auto-merge)
4	GitHub Copilot	Agentic suggestions	Documented only
5	GPT-4 / Cursor	Comparative refactoring options	Manual corrections applied

8.3 LLM Usage Evidence and Screenshots

Note on LLM Usage (Assumption) Assumption: This report documents the use of AI tools only where they were used for assistance. It does not describe or report AI usage for the specific LLM-related task defined in the assignment.

Screenshot placeholders:

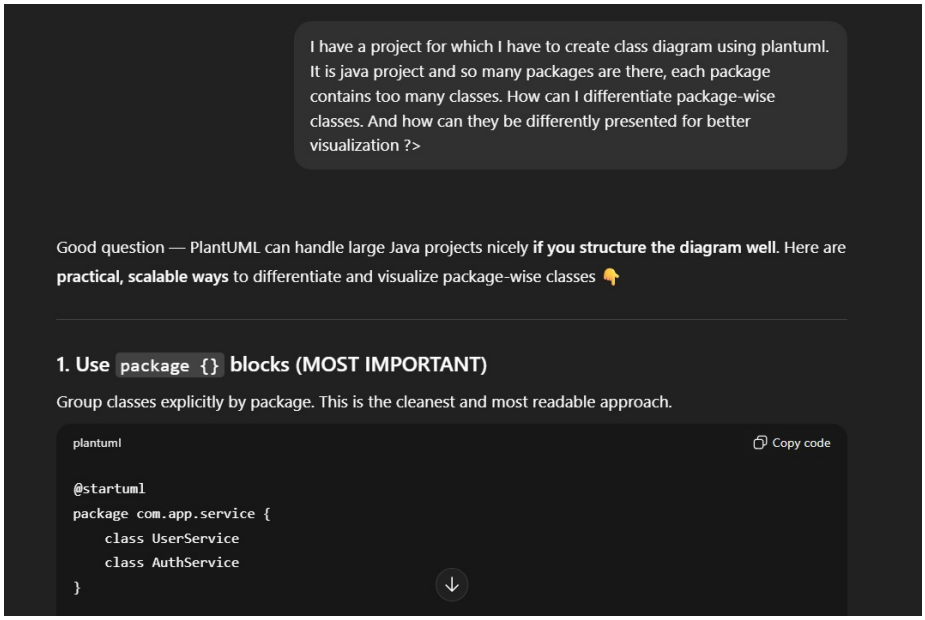


Figure 5: Task 1 PlantUML guidance 1

Task 1

- **Screenshot label:** Task 1 - PlantUML visualization guidance / ChatGPT 5.2 / (attached)
- **Prompt (summary):** How to differentiate package-wise classes in PlantUML for a large Java project and present them for better visualization.
- **LLM response (summary):** Practical, scalable suggestions: use `package {}` blocks to group classes by package, color-code packages for readability, collapse or hide less-relevant classes, and provide small PlantUML snippets showing package blocks and color styling.
- **Qualitative notes:** The suggestions are highly practical and correct for improving diagram readability. The response offers concise, actionable PlantUML patterns (package blocks, color labels).
- **Manual contribution:** Team applied these techniques when producing

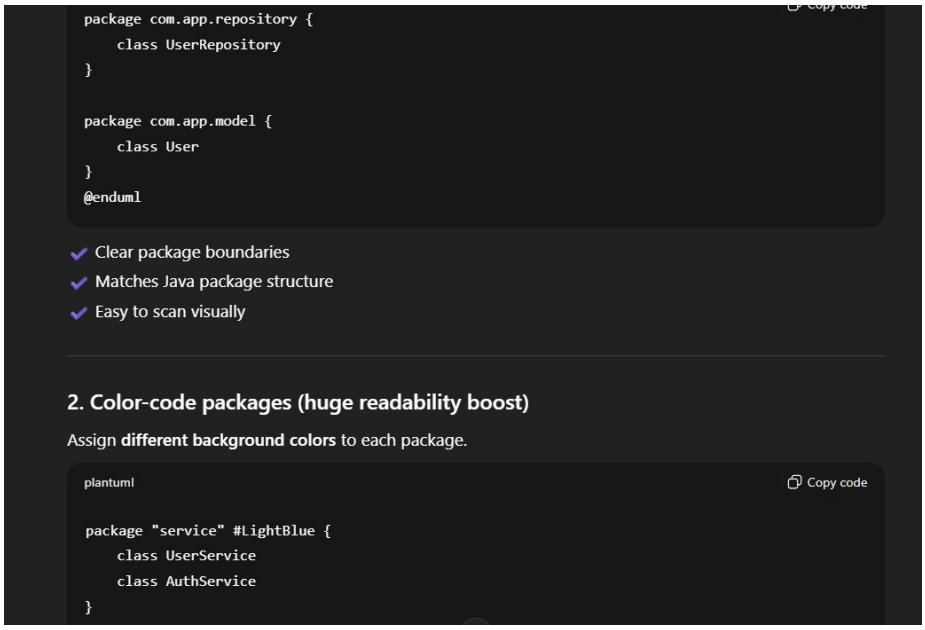


Figure 6: Task 1 PlantUML guidance 2

the subsystem diagrams in `docs/project1/Task 1/uml/` (Search, User-Role, Weblog). We selected key classes per package, applied package grouping and color-coding, and adjusted layout manually to improve readability and preserve important dependencies.

Task 2

- **Screenshot label:** Task 2 - SonarQube startup guidance / (attached)
- **Prompt (summary):** How to run SonarQube on Windows (prerequisites, directory, start command).
- **LLM response (summary):** Install Java 17 and set `JAVA_HOME`; ensure port 9000 is free; cd into `bin\windows-x86-64` and run `StartSonar.bat` as Administrator; wait for logs and open `http://localhost:9000`.
- **Qualitative notes:** Correct and actionable; remember to set `JAVA_HOME` and run with sufficient permissions.
- **Manual contribution:** Team used these steps to start SonarQube and collect scans used in Task 2 and Task 3 analyses.
- **Screenshot label:** Task 2 - SonarQube first-login / (attached)
- **Prompt (summary):** What are the default first-time login steps and credentials for SonarQube?

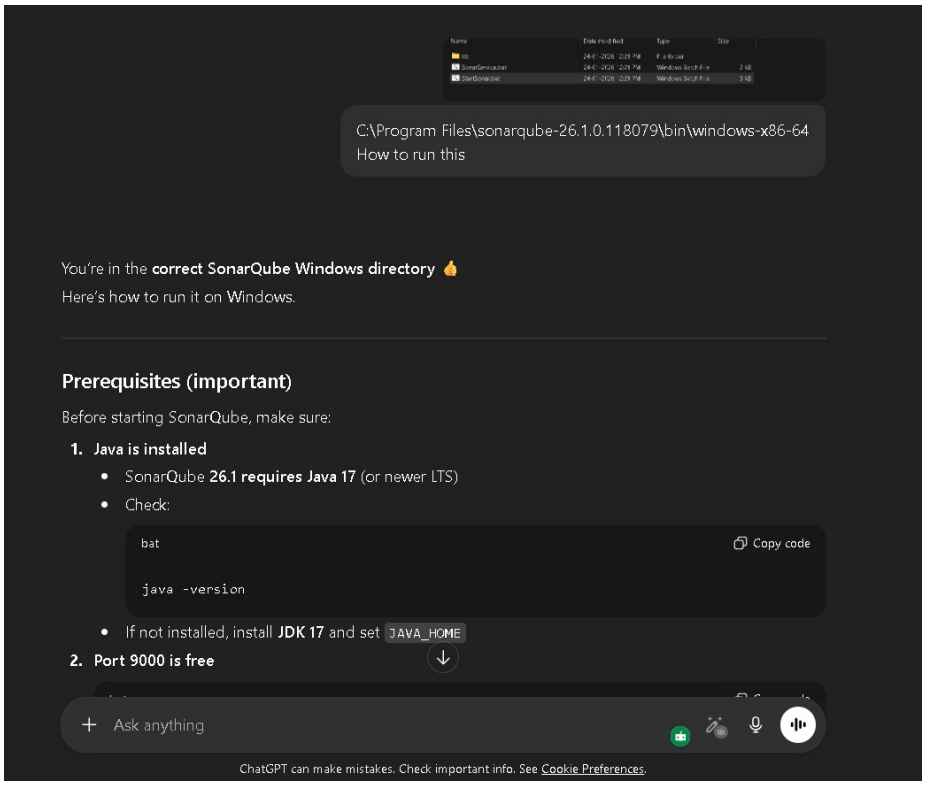


Figure 7: Task 2 SonarQube guidance 1

2. **Port 9000 is free**

```bat
netstat -ano | findstr :9000
```

How to start SonarQube (Windows)

Method 1: Using Command Prompt (recommended)

1. Open **Command Prompt as Administrator**
2. Go to the directory:

```bat
cd "C:\Program Files\sonarqube-26.1.0.118079\bin\windows-x86-64"
```

3. Start SonarQube:

```bat
StartSonar.bat
```

You should see logs scrolling and eventually:

```csharp
SonarQube is up
```

Figure 8: Task 2 SonarQube guidance 2

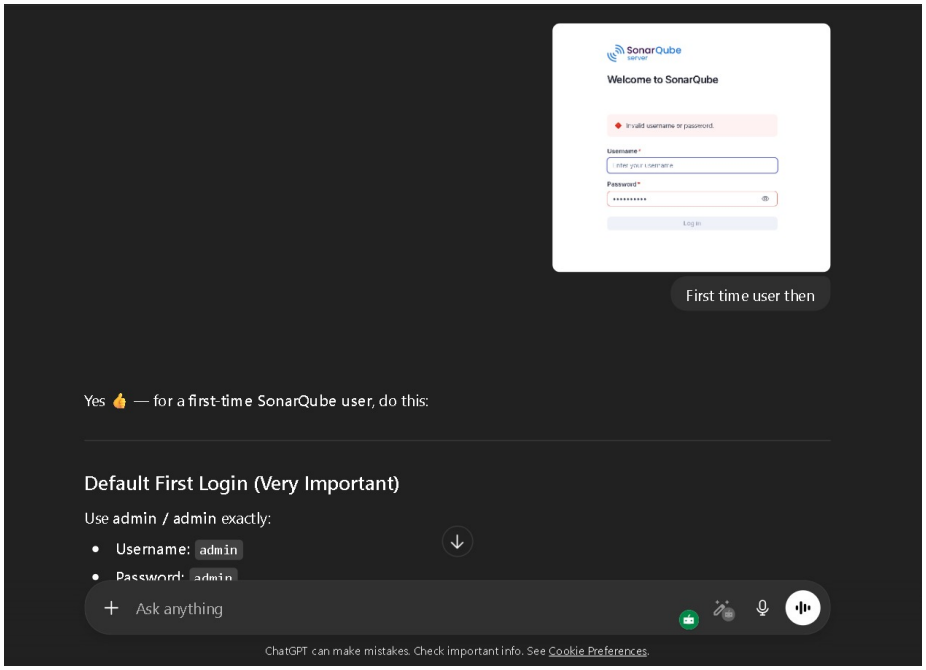


Figure 9: Task 2 SonarQube login 1

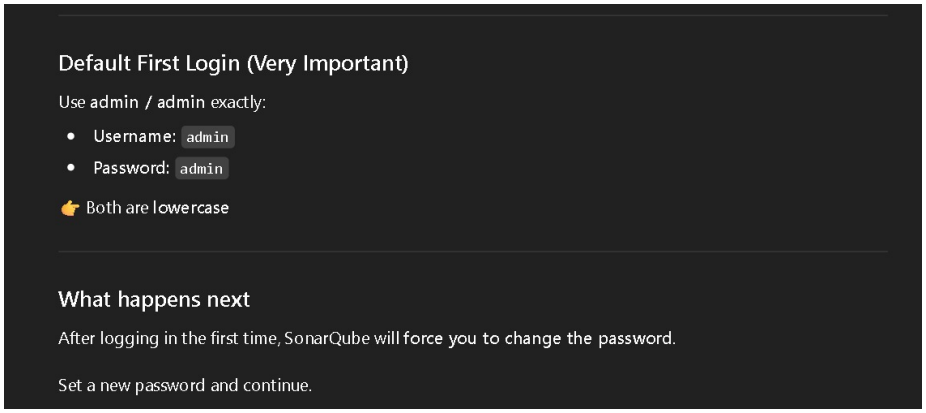


Figure 10: Task 2 SonarQube login 2

- **LLM response (summary):** Default credentials are `admin` / `admin` (both lowercase); SonarQube forces a password change on first login.
- **Qualitative notes:** Accurate; follow immediate password rotation and create a user token for CI scans.
- **Manual contribution:** Team performed first-time login, changed the admin password.

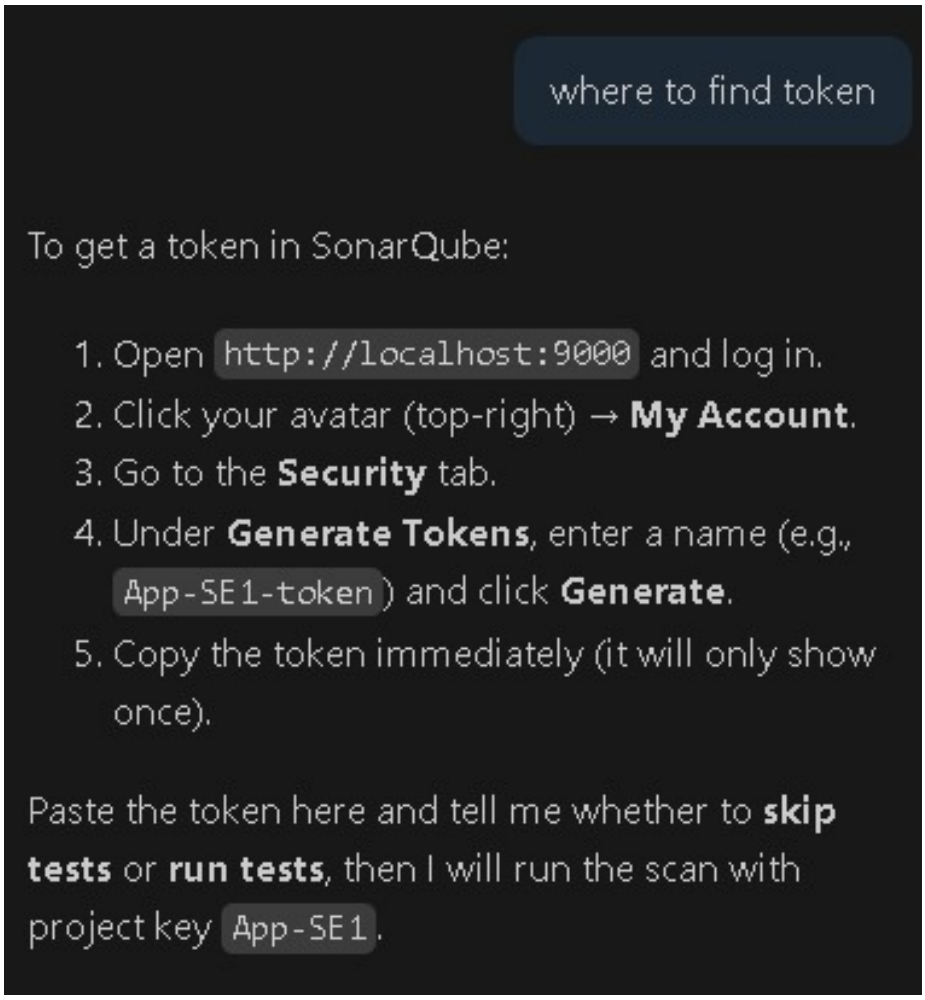


Figure 11: Task 2 SonarQube token instructions

- **Screenshot label:** Task 2 - SonarQube token location / (attached)
- **Prompt (summary):** Where to find or generate a SonarQube token for authenticated scans?

- **LLM response (summary):** Open `http://localhost:9000` → avatar (top-right) → `My Account` → `Security` → `Generate Tokens`; enter a name and click `Generate`, then copy the token immediately (it is shown only once).
- **Qualitative notes:** Correct and security-sensitive — token is displayed only once, so copy/save it securely and use tokens for CI instead of passwords.
- **Manual contribution:** Team generated a token for the project, recorded it securely, and used it in Maven scans with `-Dsonar.login=<token>` for authenticated scanning.

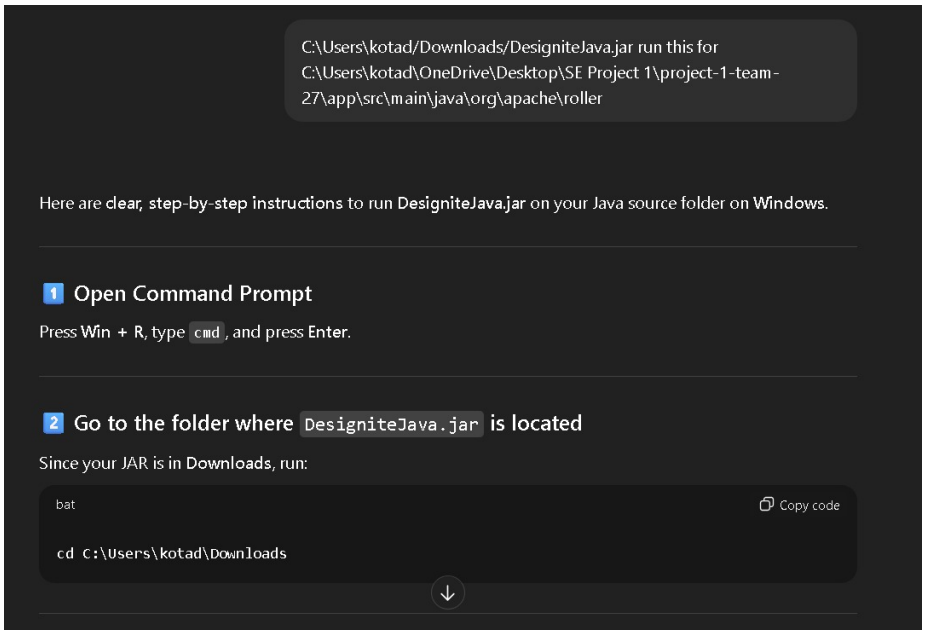


Figure 12: Task 2 Designite usage 1

- **Screenshot label:** Task 2 - Designite Java run / (attached)
- **Prompt (summary):** How to run DesigniteJava.jar on a Windows machine against a Java source folder.
- **LLM response (summary):** Run from the folder containing `DesigniteJava.jar` and use: `java -jar DesigniteJava.jar "C:\path\to\project\app\src\main\java"` (quotes needed for paths with spaces). The tool then outputs design-smell and metric reports.
- **Qualitative notes:** Accurate and actionable; remember to use quotes for paths containing spaces and to run with a JDK-compatible Java version. Inspect generated reports for smell evidence and exported metrics.

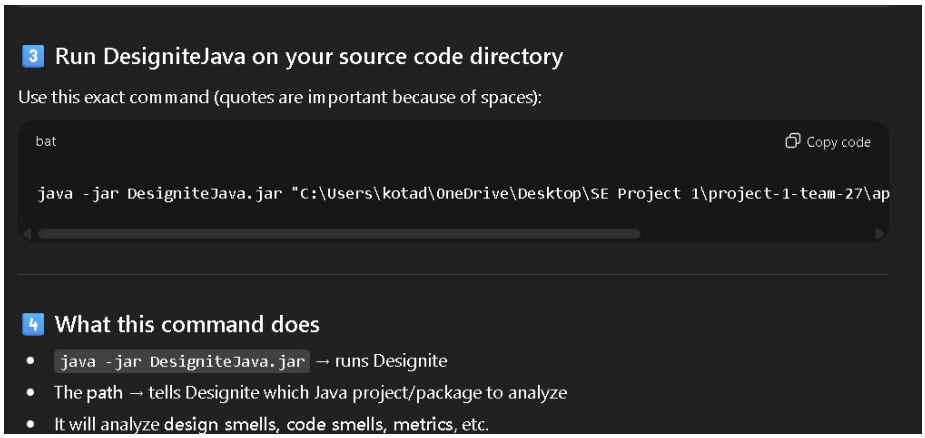


Figure 13: Task 2 Designite usage 2

- **Manual contribution:** Team ran Designite on the Roller `app/src/main/java` folder, collected the generated design smell reports and metrics, and used them as evidence for Task 2A and to prioritize Task 3 refactorings.
- **Screenshot label:** Task 2 - Checkstyle run and output / (attached)
- **Prompt (summary):** Exact commands to run Checkstyle on Windows against the project source (jar + config file + path).
- **LLM response (summary):** Commands (copy-paste ready): change to Downloads, then run: `java -jar checkstyle-13.0.0-all.jar -c checkstyle.xml "C:\Users\...\project-1-team-27\app\src\main\java" > checkstyle-report.txt` (use ^ for Windows line continuation in long commands). The report file contains rule violations and warnings.
- **Qualitative notes:** Correct and immediately usable; output is a plain-text report listing violations (e.g., cyclomatic complexity, method length) that were used to identify refactoring hotspots.
- **Manual contribution:** Team executed Checkstyle, reviewed `checkstyle-report.txt`, and used the violations to prioritize manual refactorings in Task 3A (noting methods/classes with high complexity or length).
- **Screenshot label:** Task 2 - Imperative Abstraction definition and symptoms / (attached)
- **Prompt (summary):** What is "Imperative Abstraction" and how to recognize its symptoms in code?
- **LLM response (summary):** Defined Imperative Abstraction as a design smell where code models "how to do it" instead of domain concepts; symp-

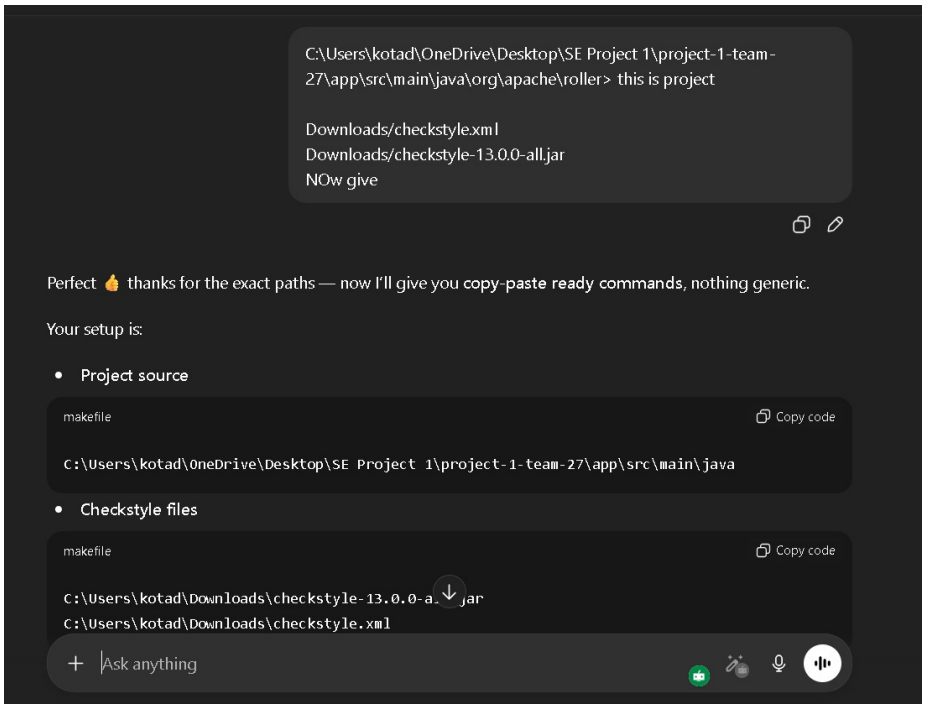


Figure 14: Task 2 Checkstyle run 1

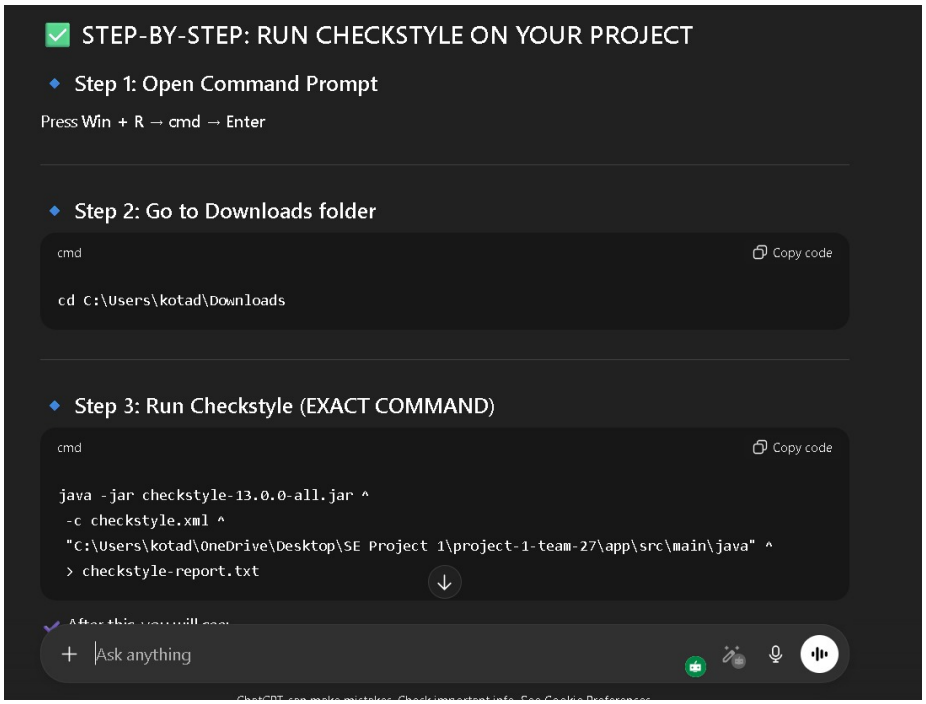


Figure 15: Task 2 Checkstyle run 2

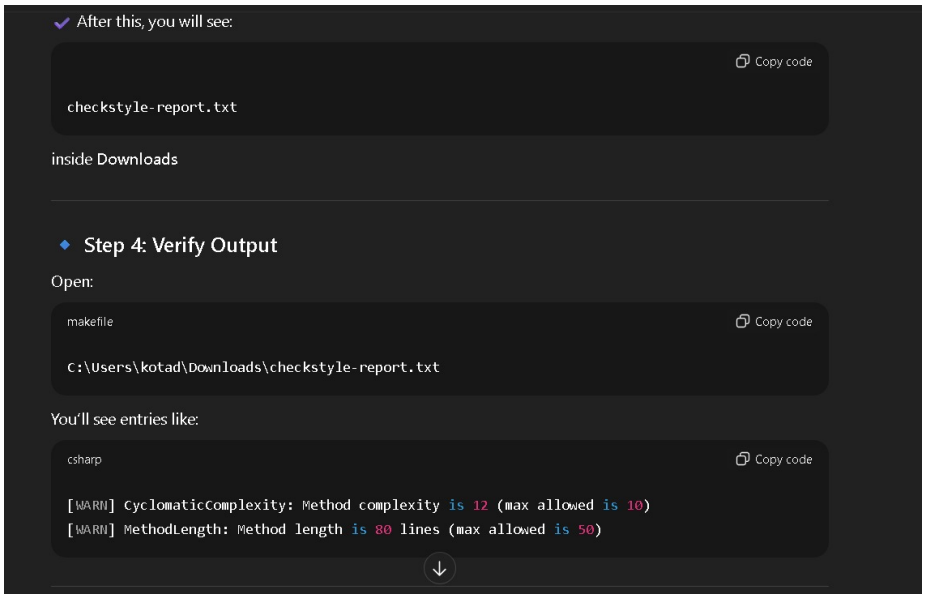


Figure 16: Task 2 Checkstyle run 3

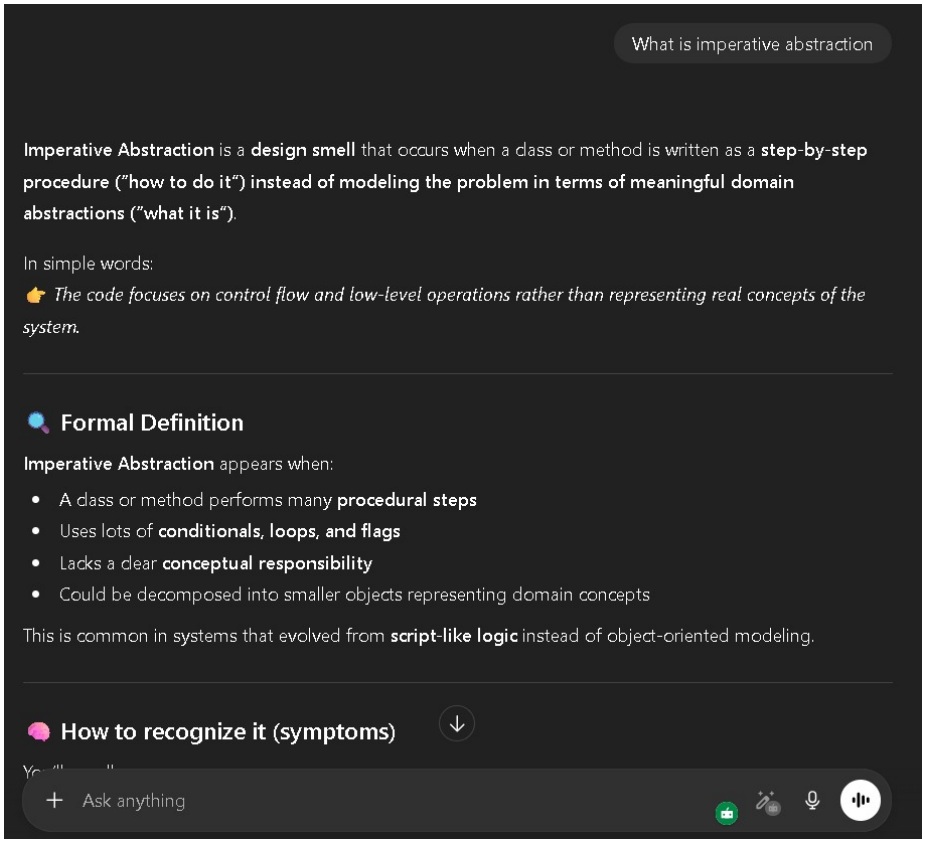


Figure 17: Task 2 Imperative Abstraction 1

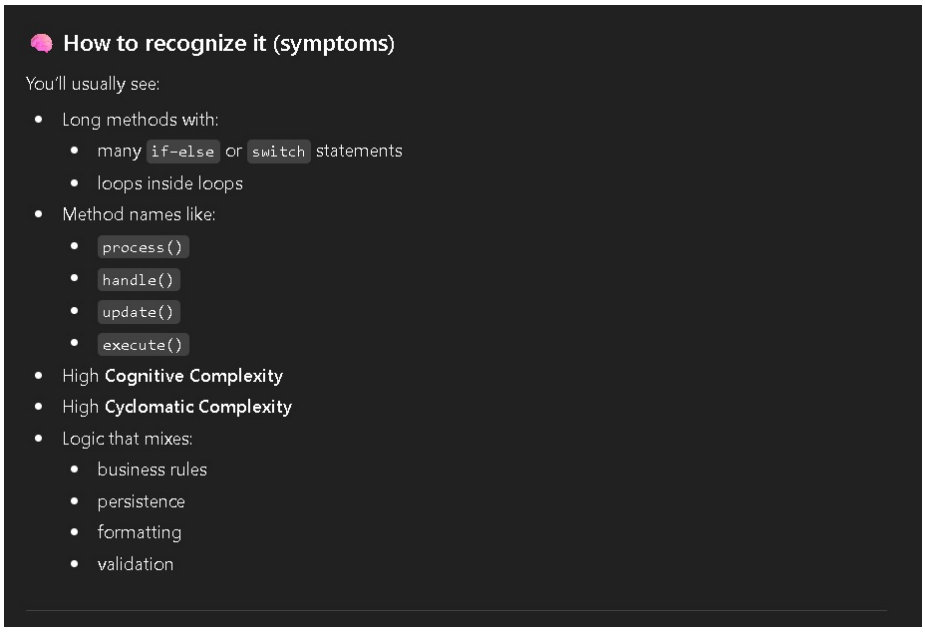


Figure 18: Task 2 Imperative Abstraction 2

toms include long methods, many conditionals/loops, nested control flow, high cyclomatic/cognitive complexity, and mixing business, persistence, and formatting logic.

- **Qualitative notes:** Accurate, clear guidance useful for mapping static-tool findings (Checkstyle/Designite) to design smells. Use it to label candidate files for refactoring but validate against runtime behavior and tests.
- **Manual contribution:** Team used these symptom definitions to confirm Imperative Abstraction occurrences (e.g., `RollerUserDetailsService` and utility classes), and guided targeted refactorings in Task 3A to extract domain abstractions and reduce method complexity.

Task 3

- **Screenshot label:** Task 3 - Gemini API key generation / (attached)
- **Prompt (summary):** How to generate a Gemini API key and set it for local use.
- **LLM response (summary):** Steps: open https://aistudio.google.com/app/apikey, log in, create or select a Google Cloud project, click `Create API key`, copy the token immediately. For local use set `GEMINI_API_KEY` as an environment variable (temporary `export GEMINI_API_KEY="your_key"` or permanent via `~/.bashrc` / system env on Windows).

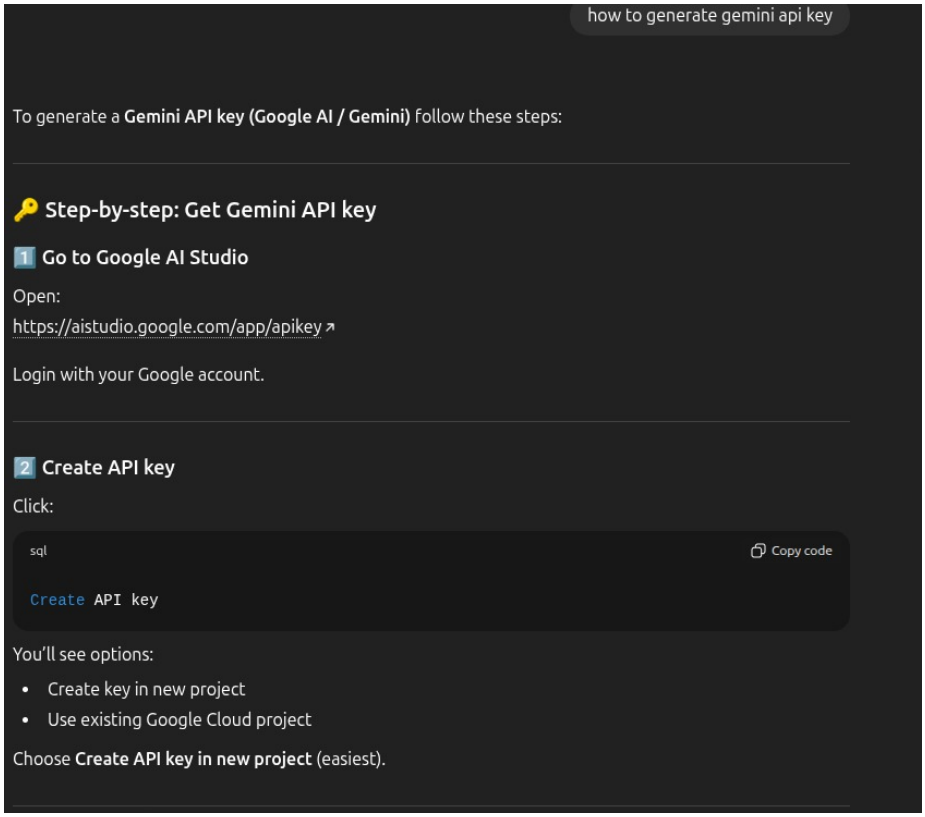
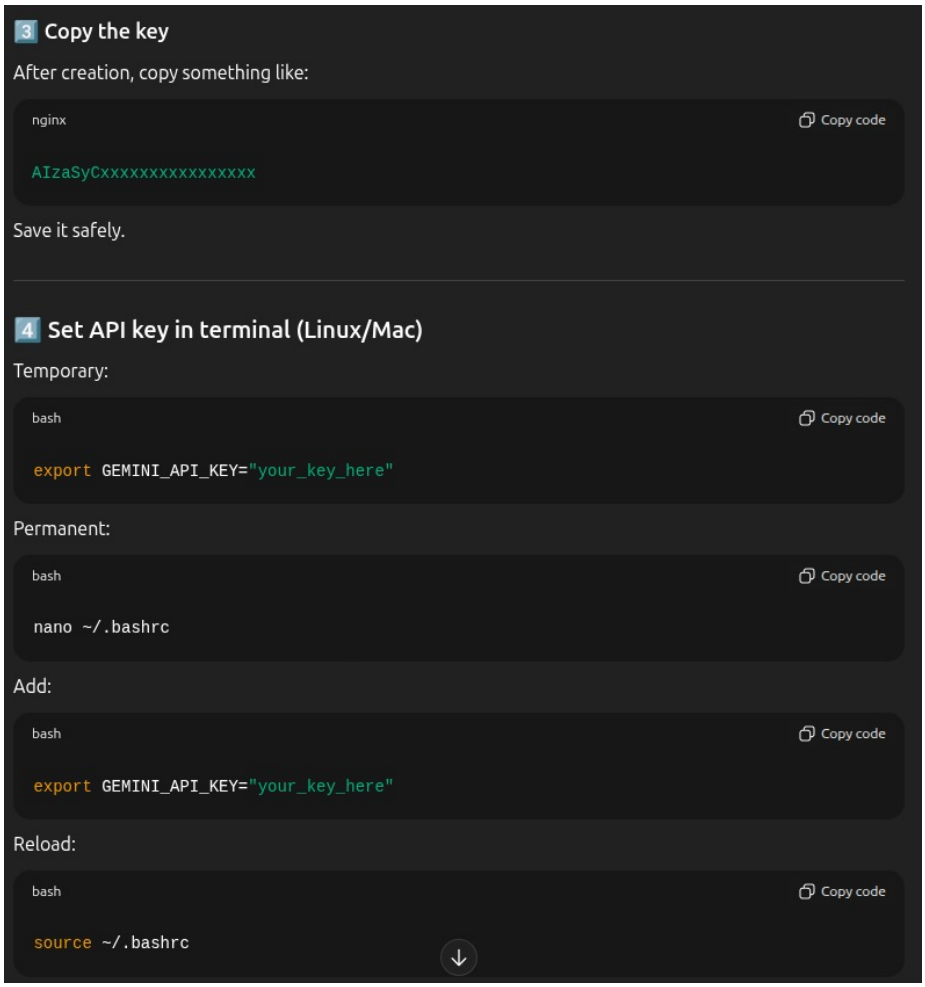


Figure 19: Task 3 Gemini API key 1



3 Copy the key

After creation, copy something like:

nginx Copy code

```
AIzaSyCxxxxxxxxxxxxxxxx
```

Save it safely.

---

4 Set API key in terminal (Linux/Mac)

Temporary:

bash Copy code

```bash
export GEMINI_API_KEY="your_key_here"
```

Permanent:

bash Copy code

```bash
nano ~/.bashrc
```

Add:

bash Copy code

```bash
export GEMINI_API_KEY="your_key_here"
```

Reload:

bash Copy code

```bash
source ~/.bashrc
```

Figure 20: Task 3 Gemini API key 2

- **Qualitative notes:** Accurate and security-sensitive; store keys securely and avoid committing them to source control. Use environment variables or CI secret storage for automated runs.
- **Manual contribution:** Team generated a Gemini API key for Task 3C experiments, stored it in a secure vault, and used it via environment variables in the local LLM-assisted refactoring pipeline documentation.

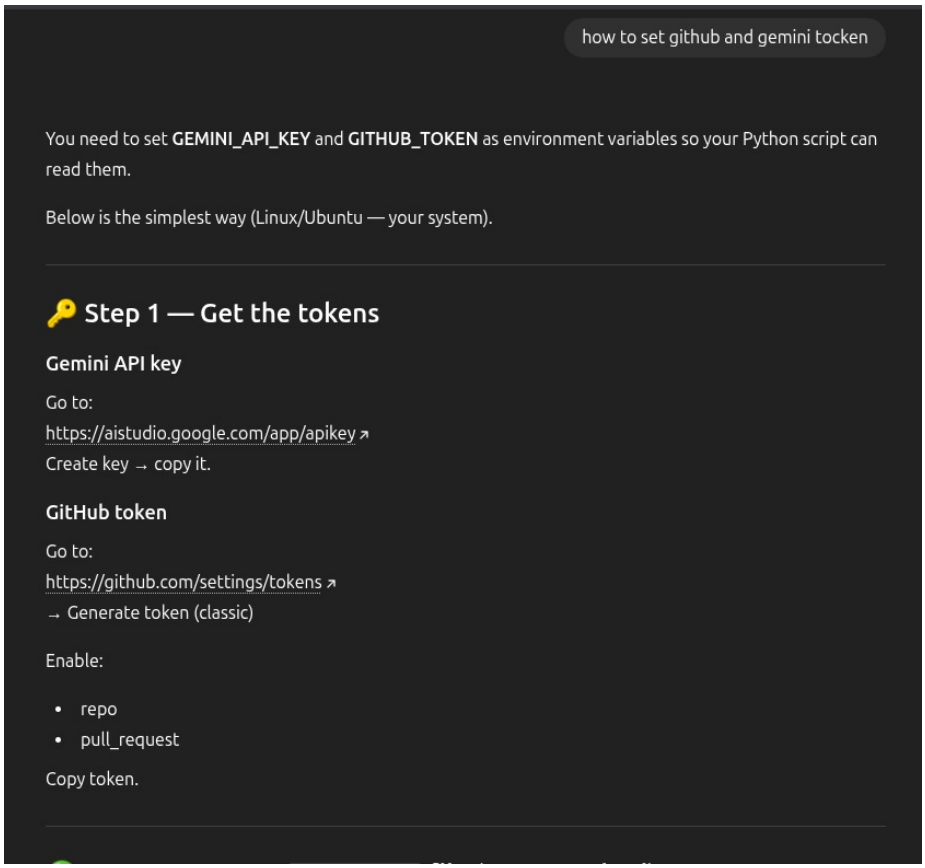


Figure 21: Task 3 API keys file 1

- **Screenshot label:** Task 3 - storing and loading API keys (`.api_keys`) / (attached)
- **Prompt (summary):** How to set `GEMINI_API_KEY` and `GITHUB_TOKEN` as environment variables and load them securely for local scripts and automation.
- **LLM response (summary):** Recommended workflow: create a secure file (e.g., `~/.api_keys`) containing `export GEMINI_API_KEY="..."` and

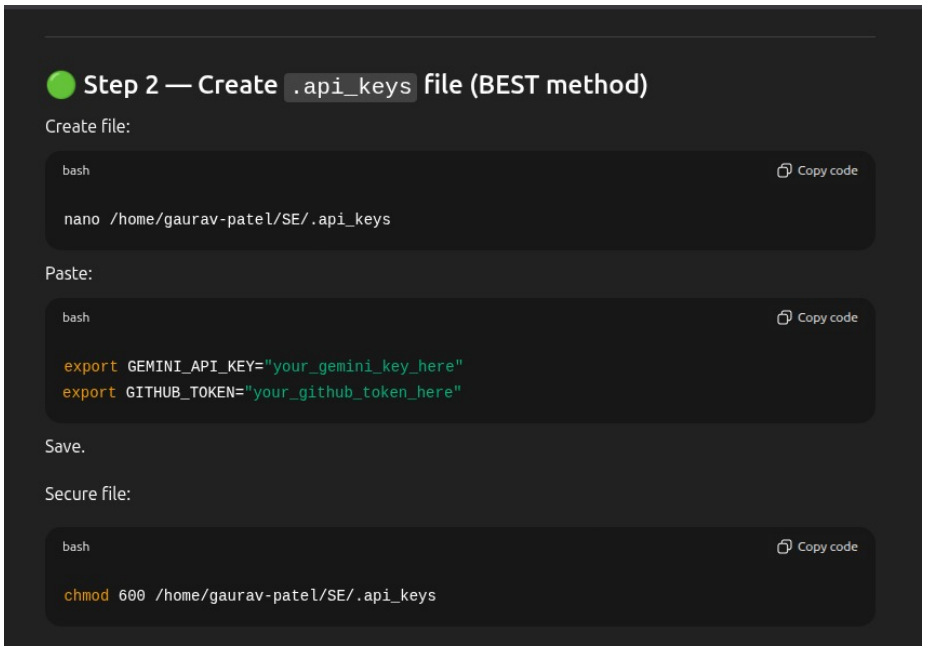


Figure 22: Task 3 API keys file 2

`export GITHUB_TOKEN="..."`; restrict file permissions (`chmod 600`); `source ~/.api_keys` in the shell or automation script; verify with `echo $GEMINI_API_KEY`; Python reads keys with `os.environ.get("GEMINI_API_KEY")`.

- **Qualitative notes:** Practical and secure for local development. For CI or shared environments prefer repository/CI secrets or vault services. Never commit key files; add them to `.gitignore`.
- **Manual contribution:** Team created a permission-restricted `.api_keys` file for local runs, added `source ~/.api_keys` to automation scripts used during experimentation, and ensured keys were stored in a secure vault and excluded from version control.

🟢 Step 3 — Load keys in terminal (test)

Run:

```bash
source /home/gaurav-patel/SE/.api_keys
```

Check:

```bash
echo $GEMINI_API_KEY
echo $GITHUB_TOKEN
```

If values print → working.

🟢 Step 4 — Python will read automatically

Your code already has:

```python
import os
GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY")
GITHUB_TOKEN = os.environ.get("GITHUB_TOKEN")
```

So nothing else needed.

🟢 Step 5 — For bash automation script

In `run_monthly.sh` add:

```bash
source /home/gaurav-patel/SE/.api_keys
```

before running Python.

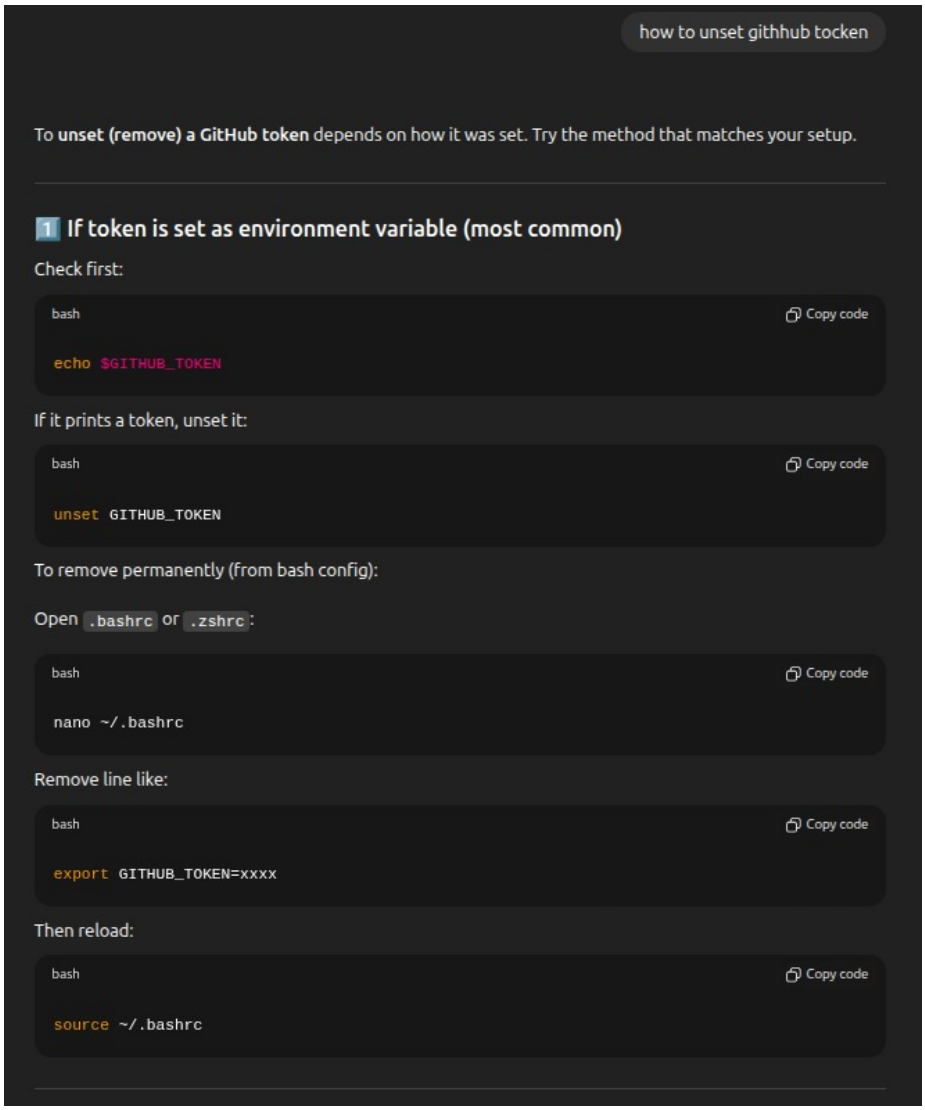
🟢 Step 6 — Temporary method (quick test)

Instead of file, you can set directly:

```bash
export GEMINI_API_KEY="xxx"
export GITHUB_TOKEN="xxx"
python3 test_single_file.py
```

But this resets when terminal closes.

Figure 23: Task 3 API keys file 3



- **Screenshot label:** Task 3 - unsetting/removing GitHub token from environment / (attached)

- **Prompt (summary):** How to unset or permanently remove a GitHub token that was set as an environment variable.
- **LLM response (summary):** For temporary removal use `unset GITHUB_TOKEN`. To remove permanently, edit `~/.bashrc` or `~/.zshrc` and delete the `export GITHUB_TOKEN=...` line, then `source ~/.bashrc` to reload the shell.
- **Qualitative notes:** Accurate and safe; double-check scripts and CI configs

for references to the token. If tokens were stored in files, remove them and update `.gitignore` to prevent future commits.

- **Manual contribution:** Team removed test tokens from local shells and CI environments following this guidance, rotated tokens when necessary, and confirmed scripts no longer expose secret values.

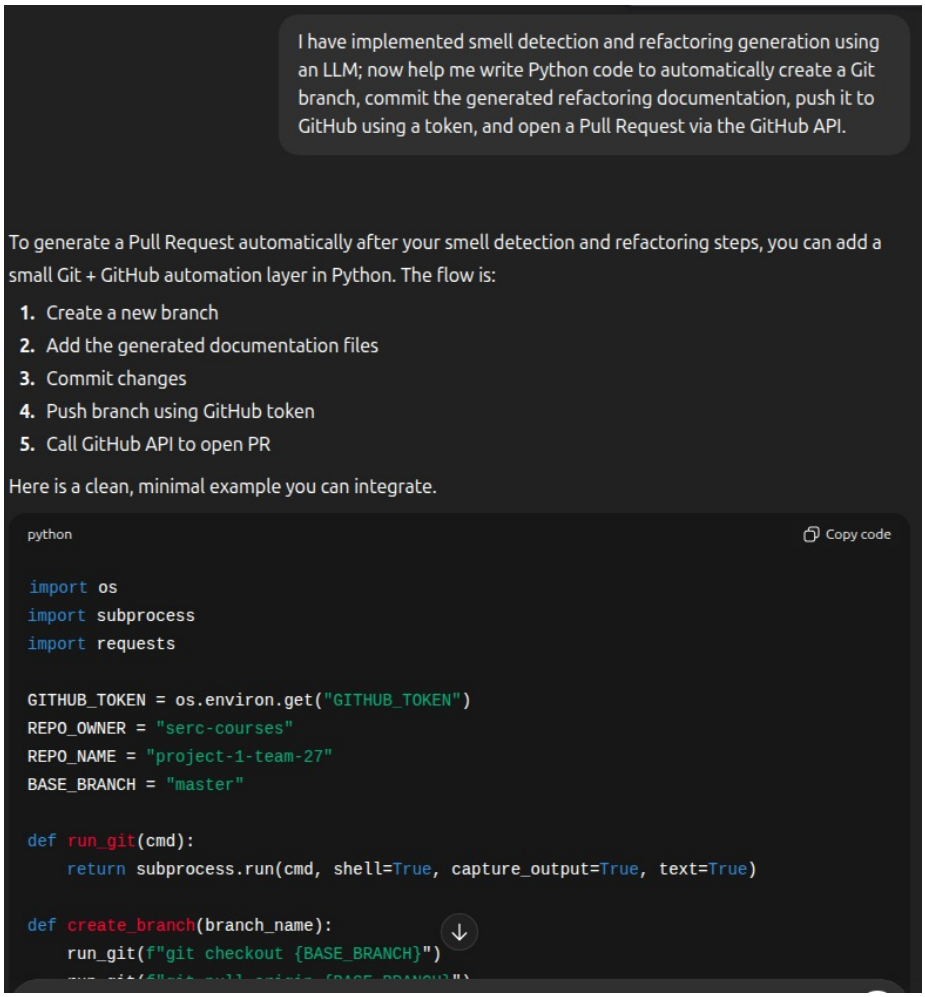


Figure 24: Task 3 GitHub PR automation 1

- **Screenshot label:** Task 3 - Git + GitHub automation (create branch, commit, push, open PR) / (attached)
- **Prompt (summary):** How to write Python automation to create a Git branch, commit generated refactoring docs, push using a token, and open a Pull Request via the GitHub API.

```python
def commit_and_push(branch_name):
    run_git("git add .")
    run_git('git commit -m "docs: add LLM refactoring analysis"')
    run_git(f"git push -u https://{GITHUB_TOKEN}@github.com/{REPO_OWNER}/{REPO_NAME}.git {

def create_pr(branch_name, title, body):
    url = f"https://api.github.com/repos/{REPO_OWNER}/{REPO_NAME}/pulls"
    headers = {
        "Authorization": f"token {GITHUB_TOKEN}",
        "Accept": "application/vnd.github.v3+json"
    }
    data = {
        "title": title,
        "body": body,
        "head": branch_name,
        "base": BASE_BRANCH
    }
    r = requests.post(url, headers=headers, json=data)
    print(r.json())

def generate_pr():
    branch_name = "llm-refactor-branch"

    create_branch(branch_name)
    commit_and_push(branch_name)

    pr_title = "Automated Refactoring Documentation"
    pr_body = "This PR contains LLM-generated refactoring analysis and documentation."

    create_pr(branch_name, pr_title, pr_body)

if __name__ == "__main__":
    generate_pr()
```

Figure 25: Task 3 GitHub PR automation 2

- **LLM response (summary):** Provided a minimal Python example using `subprocess` for Git commands and `requests` for the GitHub REST API: create branch from base, `git add/commit`, push with `https://<TOKEN>@github.com/...` URL, and POST to `https://api.github.com/repos/{owner}/{repo}/pulls` with `Authorization: token <TOKEN>` to open a PR.
- **Qualitative notes:** Practical and runnable example; ensure the token has `repo` and `pull_request` scopes, avoid logging tokens, and prefer using Git remote credential helpers or CI secrets instead of embedding tokens in commands. Validate the pushed branch name and PR metadata before opening production PRs.
- **Manual contribution:** Team integrated the example into the Task 3C pipeline documentation, adapted the script to our repository (`serc-courses/project-1-team-27`), tested PR creation on a forked/testing repo, and required manual review before merging — no automated merges were performed.

8.4 LLM Output Quality and Manual Contributions (short)

- LLMs provided runnable guidance (diagrams, tool commands, smell definitions, token handling, automation snippets).
- The team executed and verified tool outputs and applied only manually reviewed refactorings; agentic and automated outputs remained documented/testing-only.
- Secrets were handled securely (permissioned files, vaults, rotation); no secret data was committed.

8.5 LLM Usage Summary (brief)

- LLMs sped up discovery and produced practical artifacts; all outputs were advisory and validated by the team.
- Security and reproducibility were prioritized: tokens secured, automation tested on forks, and evidence captured in Section 8.3.

---

9. Assumptions and Simplifications

1. UML diagrams focus on key classes per subsystem rather than full project-wide coverage.
2. Search and indexing analysis assumes Lucene as the primary search engine without modeling alternative implementations.

3. Metrics are based on tool outputs at the time of analysis (Checkstyle, Designite, CK) and may vary with configuration.
4. Post-refactoring metrics were collected on the refactoring branch noted in Task 3B.
5. Refactoring changes were designed to preserve behavior; only manual refactorings were applied to the codebase.
6. Agentic and LLM refactorings (Tasks 3C and 4) are documented only and not merged into production code.
7. LLM comparisons in Task 5 follow the single-prompt constraint per smell.

---

10. Testing and Validation

- Build command: mvn install
- Status: BUILD SUCCESS (Task 3A documentation)

---

11. Team Contributions

Team contributions and roles:

Member	Key Contributions
Akshat (2025201005)	Conducted design smell analysis for Task 2A, compiled code metrics for Task 2B, carried out manual refactoring for Task 3A and refactoring metrics for Task 3B. Created final report. Assisted in Task 1 class diagram
Om (2025201008)	Created the Task 1 class diagram, assisted in Task 2A design smell analysis, carried out manual refactoring for Task 3A. Created final report.
Hardik (2025201046)	Performed the Task 1 analysis comparing LLM and manual approaches and contributed to the Task 5 comparison (manual vs LLM vs agentic). Carried out manual refactoring in task 3A. Created final report.
Parv (2025201093)	Prepared the documentation for Task 1, contributed to Task 2A design smell analysis, manual refactoring for Task 3A and completed Task 4 (agentic refactoring).
Gaurav (2025201065)	Designed the class diagrams for Task 1, developed the automated LLM pipeline for Task 3C, contributed to Tasks 3A. Contributed to Manual vs LLM task 1 and Manual vs LLM vs Agentic report for task 5.